ARCHITECTURE WHITEPAPER CLASS MANAGEMENT SYSTEM (CLASSBUN)

Offline-First Desktop App Architecture & Sync Guide

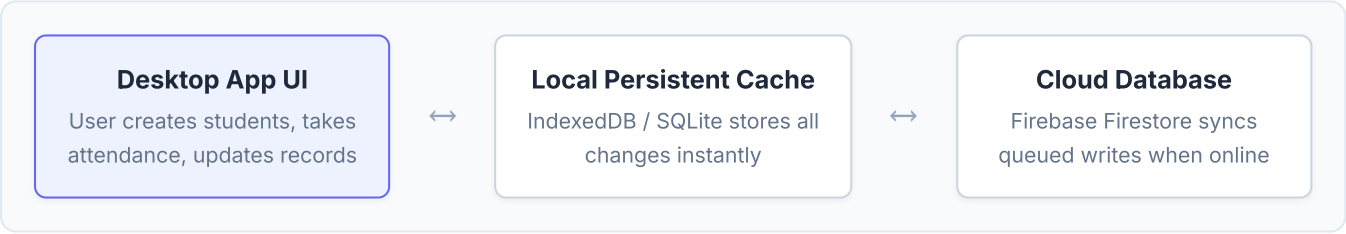
Turning a React + Vite + Firebase Web App into an Installable, Resilient Offline Desktop Application

Stack: React 19, Vite, Firebase Firestore, TypeScript **Author:** Antigravity AI Pair Programmer **Status:** Architectural Blueprint

> **Executive Summary:** Yes, this is 100% possible and is an established industry pattern known as **Local-First / Offline-First Architecture**. Your current codebase (React, TypeScript, Tailwind, Firebase) can be converted directly into an installable desktop application without discarding your existing UI code.

1. How the Offline & Online Sync Flow Operates

The core philosophy of an offline-first system is: **the local database is the single source of truth for the UI**. The network is treated as a secondary synchronization channel rather than a hard dependency.



- **When Offline:** All queries (student lists, class rosters) read directly from the local machine's disk in 0 milliseconds. Writes are confirmed immediately in the UI and stored in a pending mutation queue.
- **When Reconnecting:** The app detects the internet connection and automatically streams the pending mutation queue up to Firebase Firestore. Any remote modifications made by other users are simultaneously pulled down.

2. Packaging into a Desktop App: Tauri vs Electron

To deliver an installable desktop application (e.g. `.exe` for Windows or `.dmg` for Mac), you wrap your existing Vite build:

Criteria	Tauri (Recommended)	Electron
Installer Size	**~5 MB to 15 MB** (Extremely lightweight)	~80 MB to 150 MB (Bundles full Chromium)
Memory Usage	Very low (~30–50 MB RAM)	High (~150–300 MB RAM)
Rendering Engine	System WebView (Edge WebView2 on Windows)	Dedicated Chromium instance
Security & Auto-Updates	Built-in secure updater & tight permission model	Matured auto-updater ecosystem
Ease of Vite Integration	Native first-class support (`npm run tauri dev/build`)	Requires `vite-plugin-electron`

3. Data Synchronization Strategies

Approach A: Firestore Built-in Offline Persistence (Recommended First Step)

Firebase Firestore already features native offline synchronization. In your app's current `src/lib/firebase/config.ts`, you can enable persistent disk caching with zero changes to your existing queries or components:

```
// src/lib/firebase/config.ts
import { initializeApp } from 'firebase/app';
import {
  initializeFirestore,
  persistentLocalCache,
  persistentMultipleTabManager
} from 'firebase/firestore';

const app = initializeApp(firebaseConfig);

// Initialize Firestore with robust multi-tab persistent disk cache
export const db = initializeFirestore(app, {
  localCache: persistentLocalCache({
    tabManager: persistentMultipleTabManager()
  })
});
```

> **How Approach A handles writes while offline:** Every call to `addDoc`, `updateDoc`, or `deleteDoc` resolves locally first. The Firebase SDK places the operation in an internal transactional log. As soon as connectivity returns, Firestore sends these events in chronological order to the cloud servers.

Approach B: Dedicated Local-First Database (SQLite / RxDB / PowerSync)

For scenarios where thousands of complex records need to be queried with relational joins or where devices may be disconnected for months at a time, a dedicated local SQLite database combined with a sync engine (such as PowerSync or RxDB) can be used alongside Tauri's native SQLite plugin.

4. Critical Considerations for Seamless Offline UX

1. Offline Authentication

Firebase Authentication persists tokens in `IndexedDB` by default. When the teacher opens the app offline, the stored token logs them in seamlessly without contacting Google servers. However, ensure token refresh failures do not force-logout the user when offline.

2. Conflict Resolution (Two Users Editing Same Record)

- **Last Write Wins (LWW):** The change with the newest timestamp overwrites previous fields (standard Firestore behavior).

- **Field-Level Merges:** Updates modify specific object fields rather than replacing whole documents, minimizing collisions.

3. File & Photo Uploads (Avatars, Documents)

Firebase Storage does not natively buffer uploads offline like Firestore does. For image uploads:

- Save the selected image to local disk cache (IndexedDB or app data folder).
- Store the upload task in an "Offline Outbox" queue.
- Trigger the background upload once an online network event fires.

4. Visual Network Status Indicator

Keep the user confident by showing their current connection and sync status in the header or status bar:

```
// Example React Hook for Network Status
import { useState, useEffect } from 'react';

export function useNetworkStatus() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);

  useEffect(() => {
    const pingOnline = () => setIsOnline(true);
    const pingOffline = () => setIsOnline(false);

    window.addEventListener('online', pingOnline);
    window.addEventListener('offline', pingOffline);
    return () => {
      window.removeEventListener('online', pingOnline);
      window.removeEventListener('offline', pingOffline);
    };
  }, []);

  return isOnline;
}
```

5. Step-by-Step Implementation Roadmap

1. **Step 1 (Immediate):** Enable Firestore `persistentLocalCache` in your web configuration to test offline querying directly in the browser's developer tools (Network tab set to "Offline").
2. **Step 2:** Add connection awareness UI (e.g. a small status dot: *"Online & Synced"* vs *"Working Offline (Changes Saved Locally)"*).
3. **Step 3:** Initialize **Tauri** in your repository (`npm install -D @tauri-apps/cli` & `npx tauri init`).
4. **Step 4:** Configure the window title, icons, and native installer settings in `src-tauri/tauri.conf.json` .
5. **Step 5:** Run `npm run tauri build` to produce your standalone Windows `.exe` / `.msi` installer!

> **Key Takeaway:** You do not need to pause your current feature development. Because you are building on standard web technologies, the transition to an offline desktop application can be performed smoothly at any stage of the project.